



Gesture Controlled Android Game

Presented by Team **Dominator069**

Members:

- Rishabh Goyal
- Navneet Gupta
- Shardul Joshi
- Rakshit Aggarwal

Introduction & Objectives

We are developing an AI-powered, gesture-controlled racing game for Android using the Qualcomm Innovation Development Kit (QIDK).

AI-Powered Control

Replaces traditional controls with hand gesture recognition for an interactive experience.

Key Objectives

- Collect & preprocess gesture data.
- Train & fine-tune a recognition model.
- Integrate into an Android racing game.
- Map gestures to game controls.

Data Collection & Model Training

Data Collection: Done

- Developed pipeline for capturing gesture samples.
- MediaPipe extracts hand landmarks (input features).
- Collected 2000+ samples per gesture across 8 gestures.
- Dataset split: 70% training, 30% test.

Model Training: Done

- Trained TensorFlow classification model.
- Converted to TensorFlow Lite (TFLite) for mobile.
- Confirmed TFLite model runs correctly on PC.
- Computed confusion matrix and loss curve.
- Achieved solid baseline accuracy distinguishing gestures.

Model Architecture

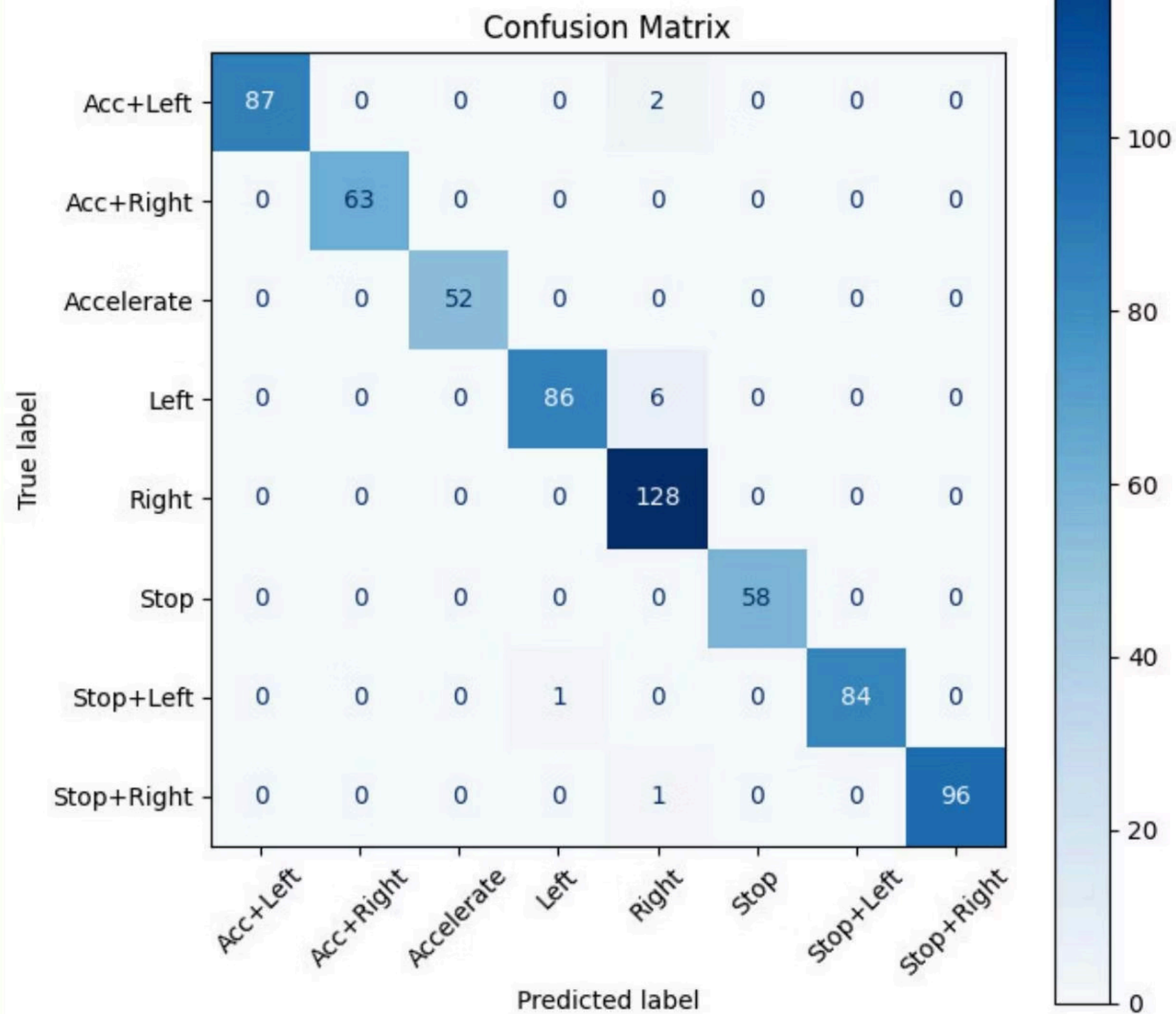
Our sequential model architecture processes hand landmark data through multiple layers for gesture classification.



Model Evaluation: Confusion Matrix

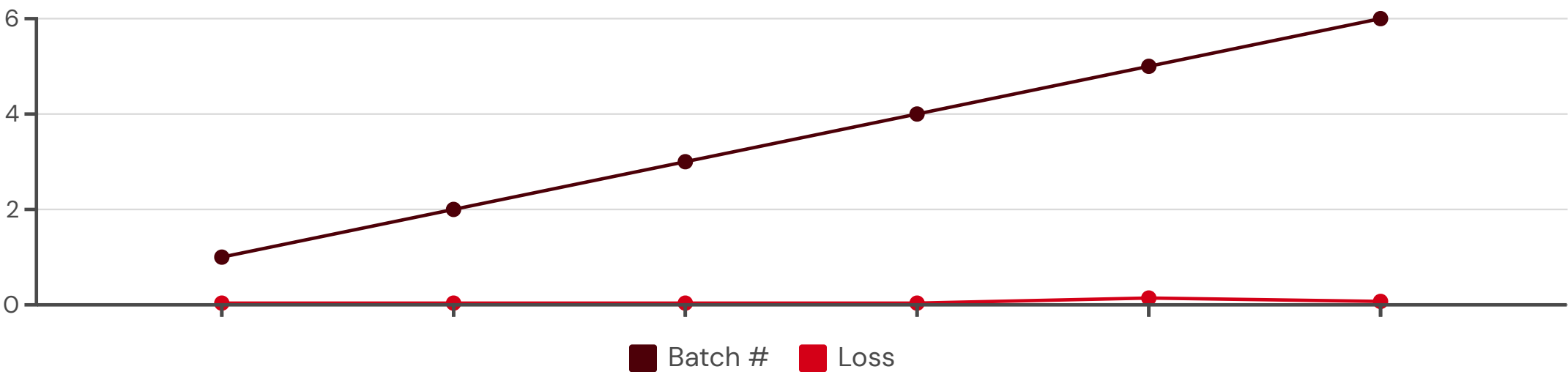
The confusion matrix illustrates our model's ability to correctly classify different hand gestures.

Acc+Left	87	0	0	0	2	0	0	0
Acc+Right	0	63	0	0	0	0	0	0
Accelerate	0	0	52	0	0	0	0	0
Left	0	0	0	86	6	0	0	0
Right	0	0	0	0	128	0	0	0
Stop	0	0	0	0	0	58	0	0
Stop+Left	0	0	0	1	0	0	84	0
Stop+Right	0	0	0	0	1	0	0	96



Loss Curve & Classification Report

Cross-Entropy Loss per Batch



Loss decreases, then peaks at Batch 5 before decreasing again.

Classification Report

Metric	Precision	Recall	F1-Score	Support
Acc+Left	1	0.98	0.99	89
Acc+Right	1	1	1	63
Accelerate	1	1	1	52
Left	0.99	0.93	0.96	92
Right	0.93	1	0.97	128
Stop	1	1	1	58
Stop+Left	1	0.99	0.99	85
Stop+Right	1	0.99	0.99	97
Accuracy	0.98			664
Macro Avg	0.99	0.99	0.99	664
Weighted Avg	0.99	0.98	0.98	664

High precision, recall, and F1-scores across all gestures, with an overall accuracy of 98%.

Classification Report – Expanded Dataset

Overall Performance Metrics

85.76%

Overall Accuracy

Across all gesture classes

0.88

Macro Avg. Precision

Balanced positive
predictions

0.85

Macro Avg. Recall

Correctly identified positives

0.86

Macro Avg. F1-Score

Harmonic mean of P & R

45,212

Total Samples

Robust dataset size

Key Insights from Dataset Expansion

- Dataset expanded from 2,000+ to 45,000+ samples (22x increase).
- Overall accuracy: 85.76% (slight decrease from 98% due to real-world variability).
- Best performing gesture: Accelerate (F1: 0.9318, Recall: 0.9728).
- Most challenging gesture: Stop+Right (F1: 0.7943, Recall: 0.7245).
- Larger dataset captures more real-world variations.

Classification Report – Expanded Dataset

Per-Class Metrics Breakdown

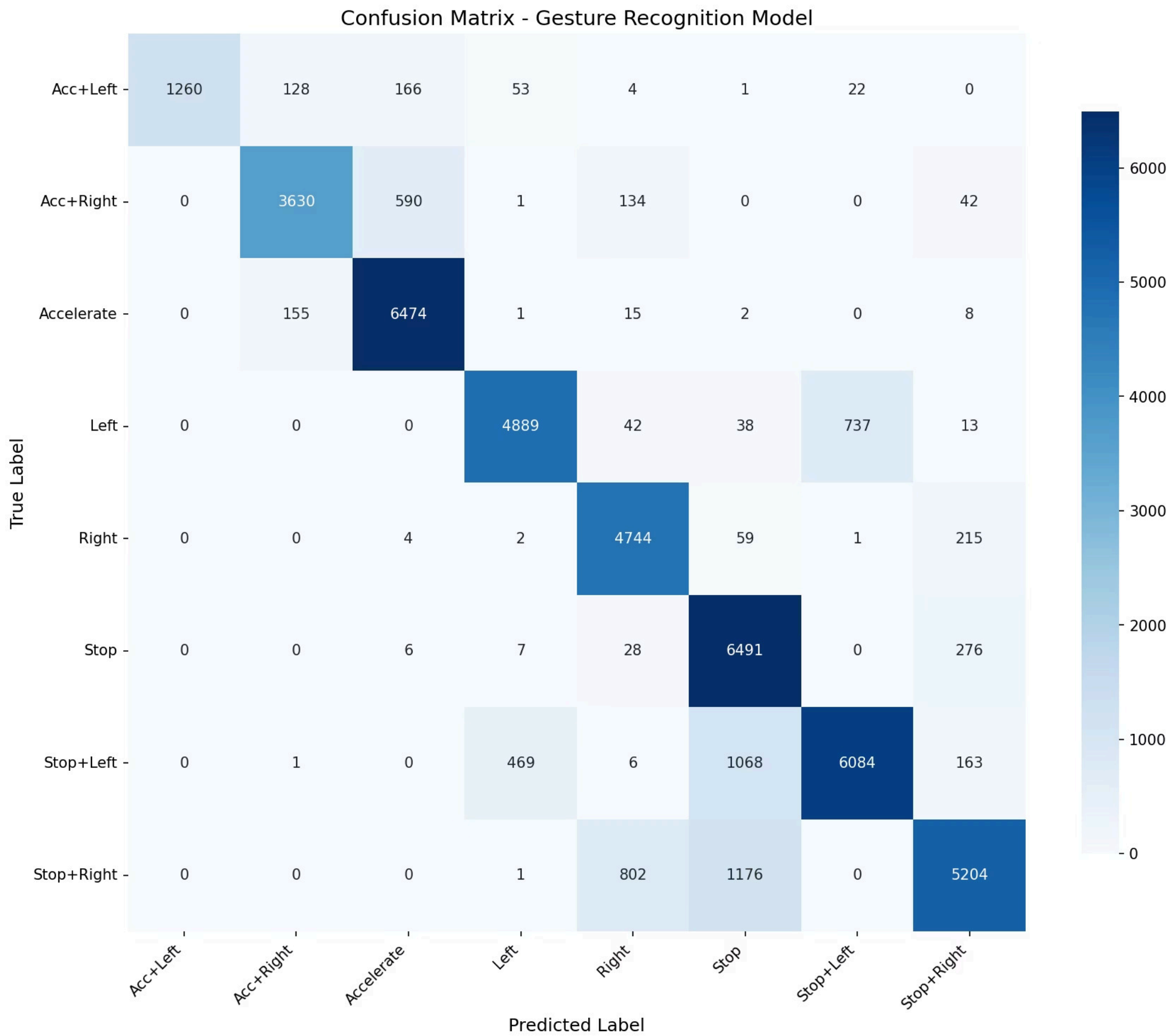
Gesture Class	Precision	Recall	F1-Score	Support
Acc+Left	1.0000	0.7711	0.8708	1634
Acc+Right	0.9274	0.8256	0.8735	4397
Accelerate	0.8942	0.9728	0.9318	6655
Left	0.9015	0.8549	0.8776	5719
Right	0.8215	0.9441	0.8785	5025
Stop	0.7347	0.9534	0.8299	6808
Stop+Left	0.8890	0.7809	0.8314	7791
Stop+Right	0.8789	0.7245	0.7943	7183

This detailed breakdown by gesture reveals key performance characteristics:

- Precision across most gestures remains strong, ranging from 0.82 to 1.0.
- The 'Accelerate' gesture is classified most effectively, achieving the highest F1-score of 0.9318.
- 'Stop+Right' presents the greatest classification challenge, with the lowest F1-score of 0.7943.
- Despite variations, the model demonstrates robust classification, maintaining good accuracy across all gesture types.

Confusion Matrix Analysis – Expanded Dataset

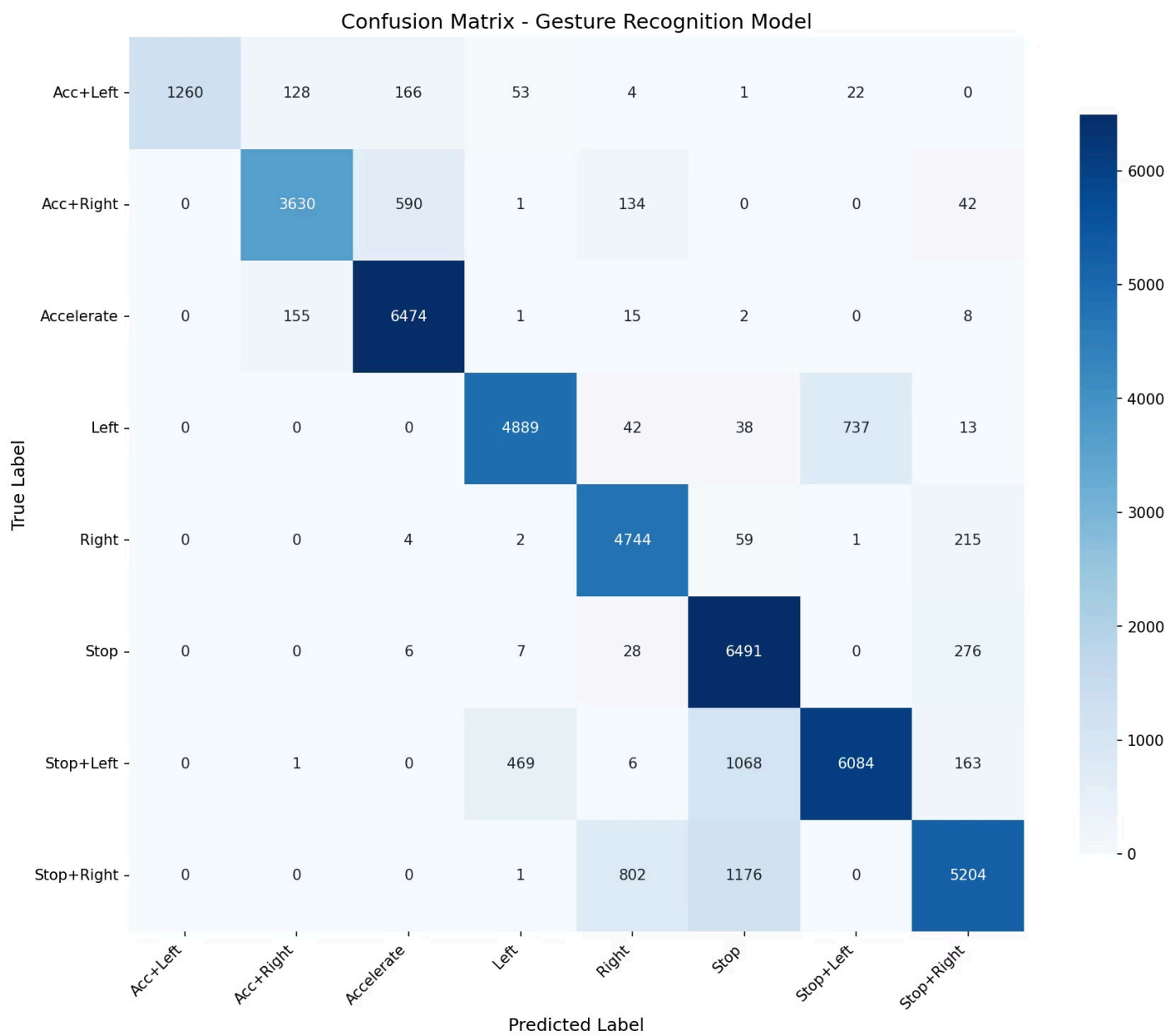
Raw Confusion Matrix – Absolute Counts



Visualizing the absolute counts of correct and incorrect classifications for each gesture.

Confusion Matrix Analysis – Expanded Dataset

Normalized Confusion Matrix – Classification Percentages

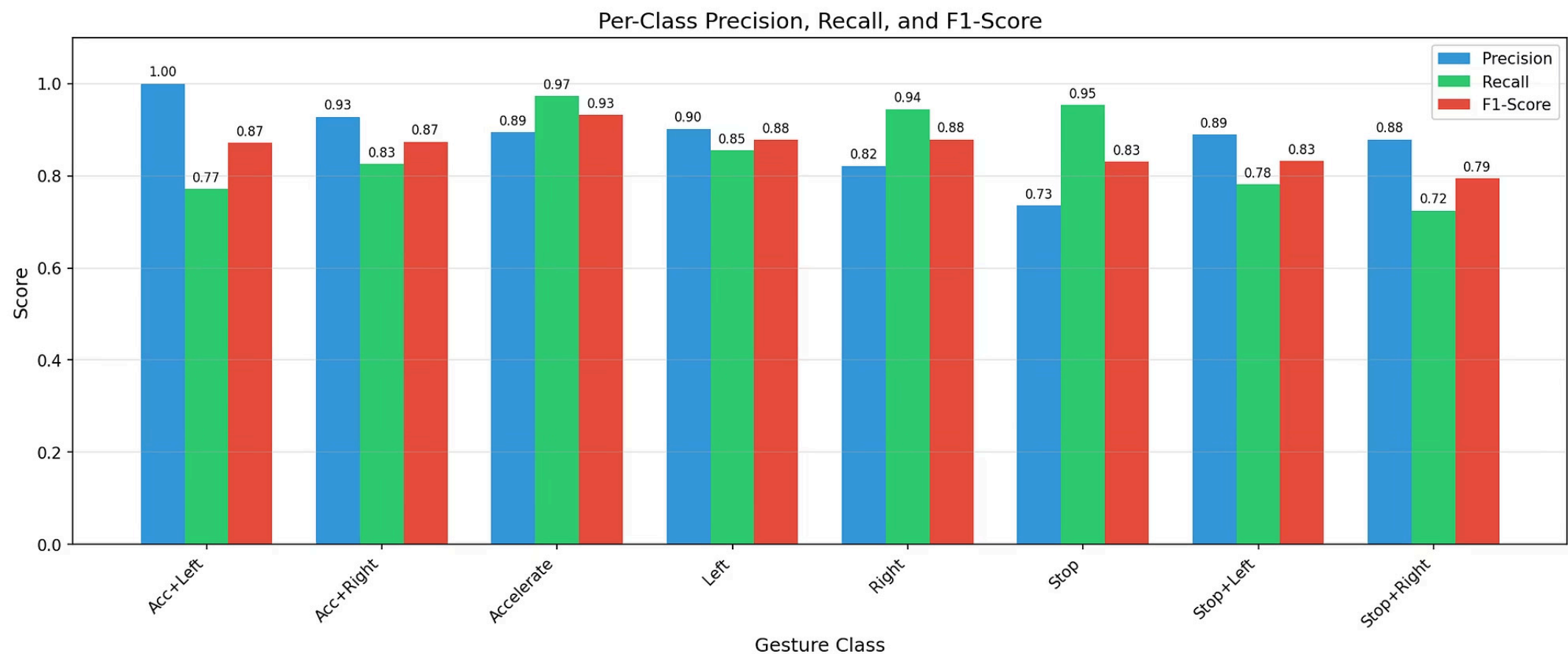


This normalized view reveals classification accuracy and misclassification patterns as percentages for clearer insights.

Diagonal values represent correct classifications, while off-diagonal values show misclassifications, highlighting specific areas for model improvement.

Confusion Matrix Analysis – Expanded Dataset

Per-Class Metrics – Precision, Recall, and F1-Score

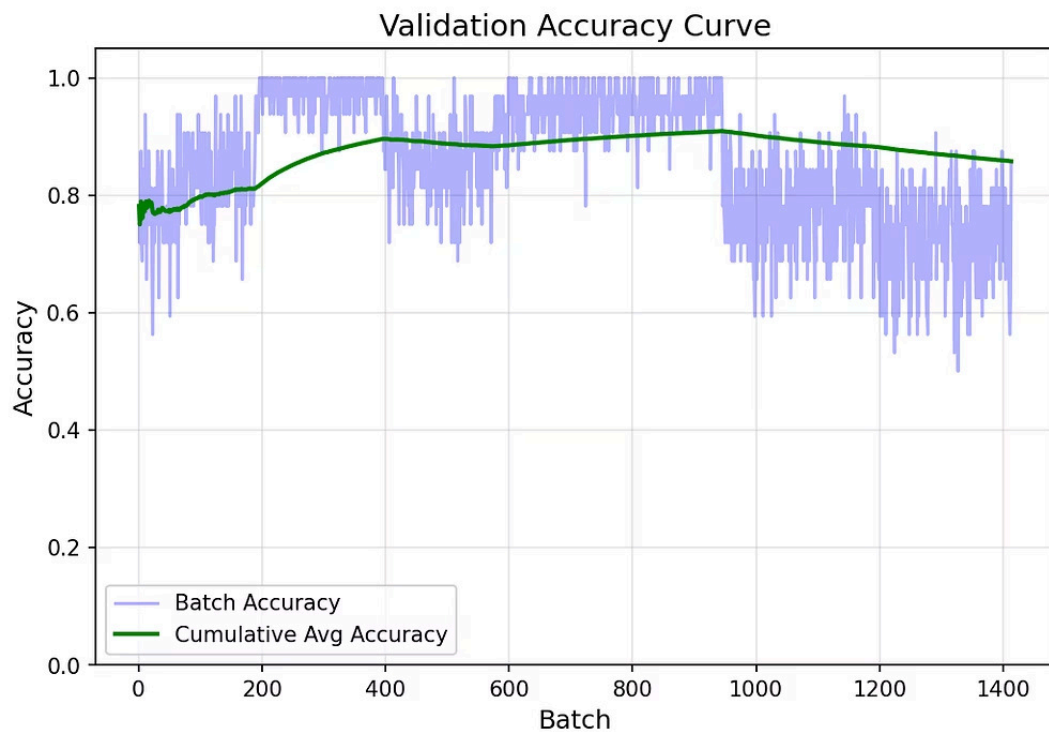
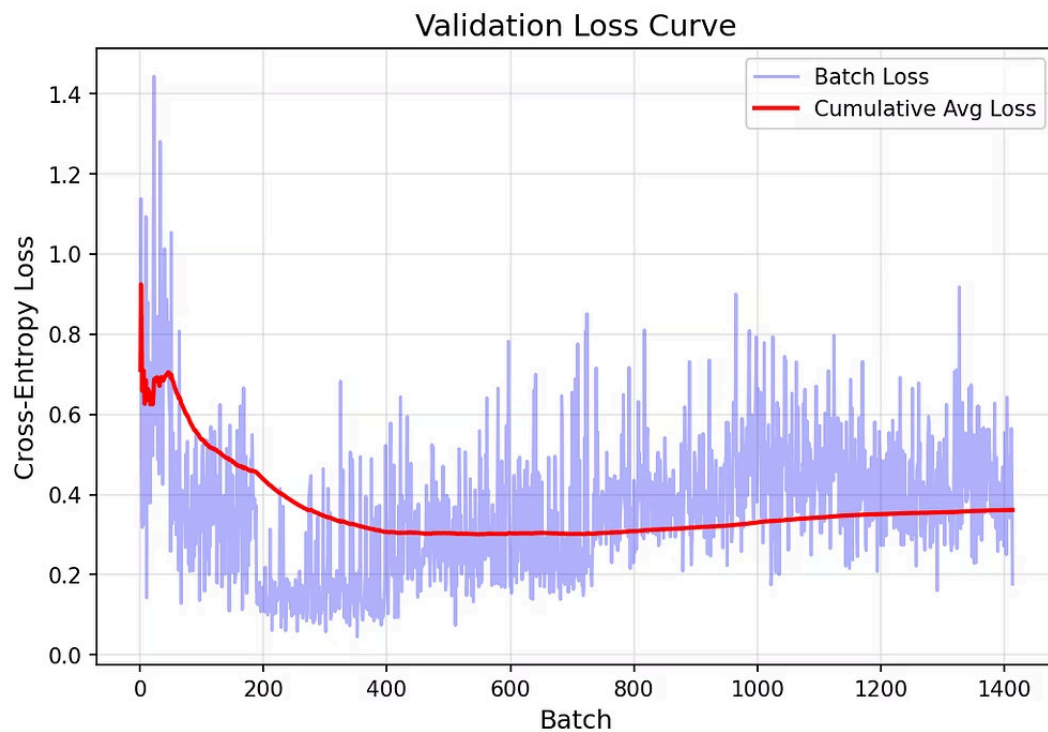


A comparative view of precision, recall, and F1-score across all recognized gestures.

This chart reveals which gestures perform well and which need improvement, guiding future model refinement efforts.

Loss Curve & Classification Report

Cross-Entropy Loss per Batch



Confusion Matrix Analysis – Expanded Dataset

Key Observations and Insights

Diagonal Dominance

Strong diagonal values confirm good classification accuracy overall, demonstrating robust model performance.

Accelerate Gesture

Achieves the highest recall (97.28%), making it our most reliably recognized gesture among all classes.

Stop+Right Challenge

Exhibits the lowest recall (72.45%), indicating it is frequently confused with other gestures.

Acc+Left Profile

Shows perfect precision (100%) but lower recall (77.11%), suggesting some missed detections despite high accuracy when identified.

Stop Gesture Nuance

Features high recall (95.34%) but suffers from lower precision (73.47%) due to a tendency for false positives.

Confusion Patterns

Misclassifications frequently arise between gestures with physically similar hand positions, posing a challenge for fine-grained distinction.

Quantization & Benchmarking

Model Conversion And Testing

Converted to DLC format and quantized to int8 and w8a16 data types.

Tested the model in 5 modes using snpe-net-run.

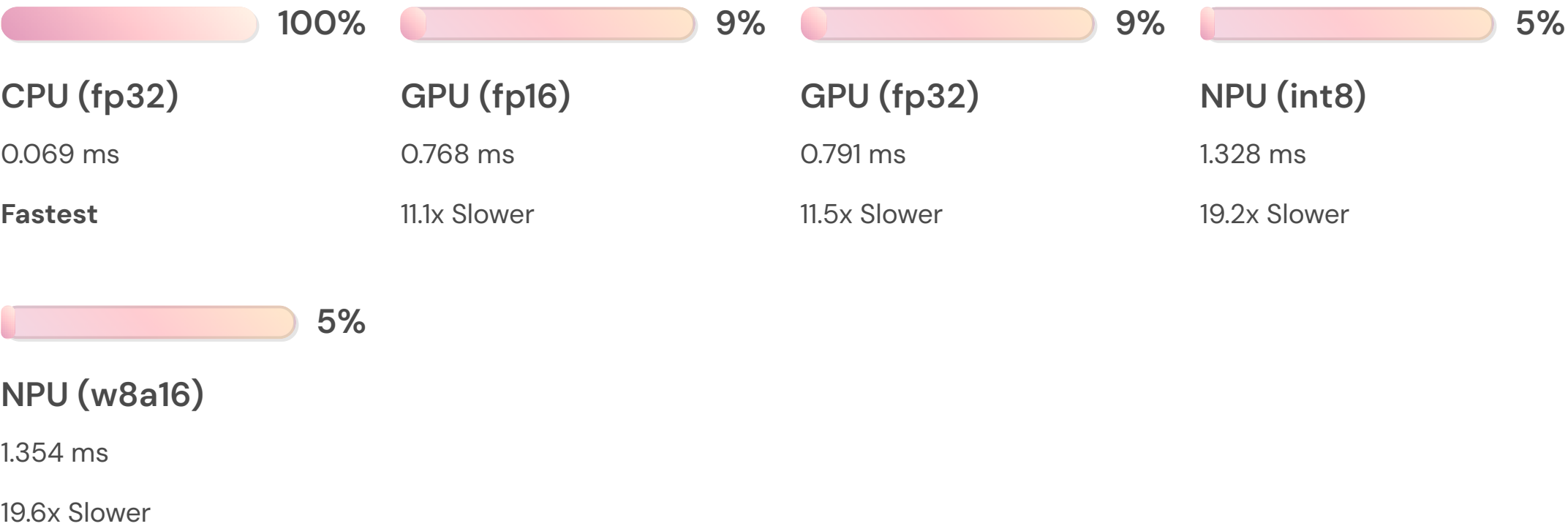
Accuracy Maintained

99.54% accuracy across all tests, confirming zero accuracy loss after quantization.

CPU Performance

CPU was unexpectedly 10x faster due to the model's lightweight nature.

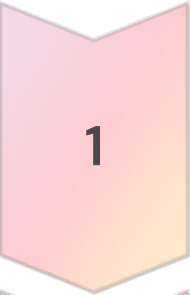
Benchmarking Report: Latency Comparison



All backends maintained **99.54% accuracy** after quantization.

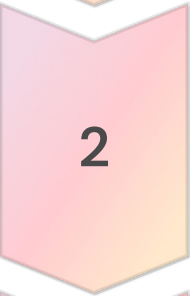
Android App: Workflow & Functionalities

Workflow



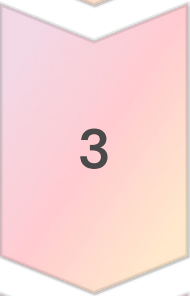
Capture Camera Feed

Obtain live video stream from the device's camera.



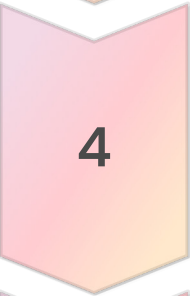
Preprocess Frames

Resize, normalize, and optimize frames for model input.



Hand Detection (MediaPipe)

Identify hand landmarks and regions using MediaPipe.



Gesture Classification (TFLite)

Classify detected hand gestures using an optimized TFLite model.



Map to Game Controls

Translate classified gestures into specific game commands.



Game Response

Execute actions and provide feedback within the game.

Key Functionalities

Camera & Display

Manage camera input (front/back, permissions) and display real-time feed with optional processing overlays.

Optimized Processing

Efficient frame handling, gesture mapping, customizable backends (CPU/GPU/NPU), low-latency loop, and model flexibility.

Debug & Development

Toggle landmark visualization and access comprehensive debugging information for development and optimization.

Qualcomm NPU Integration & Benchmarking Analysis

NPU Integration

- MediaPipe Hands successfully ran on Qualcomm NPU.
- Pre/post-processing rewritten from Python to Kotlin.
- App supports CPU, GPU, NPU (MediaPipe API), NPU (Qualcomm QNN API).
- NPU execution verified via logcat.

Our App now runs Mediapipe on NPU and the classification model on CPU as it is quite lightweight.

Benchmarking Insights

- MediaPipe framework is the primary bottleneck in the full pipeline.
- GPU delegation provides significant speedup on compatible hardware.
- High-end GPUs may perform poorly due to overhead/incompatibility.
- Lightweight classifier model is faster on CPU than NNAPI on some mobile devices.
- Performance bottleneck is heavier MediaPipe components, not the classifier.

Methodology: Qualcomm Docs + MediaPipe Adaptation for Dual-Hand Detection

- We studied Qualcomm's official documentation to understand how their AI acceleration and NPU execution works on Android hardware.
- MediaPipe's hand tracking pipeline consists of **two internal neural models**:
 - **Hand Region Detector (Model 1)**: Identifies and crops the hand region from the image.
 - **Landmark Model (Model 2)**: Predicts 21 key-point landmarks for each detected hand.
- MediaPipe by default processes **only one hand**, even if multiple hands exist in the frame.
- In the `base_app.py` file (inside `qai_utils`) — the core Mediapipe execution class — we located the logic controlling the hand loop.
- The default implementation **breaks after the first detected hand**, preventing multiple-hand processing.
- We modified this by **removing the break condition** and instead **appending all detected hands** into the output list.
- With these changes, the application now:
 - Successfully detects **two hands simultaneously**
 - Runs inference efficiently on the **Qualcomm NPU**
 - Achieves our goal of real-time dual-hand tracking

Evaluation Metrics & Methodology

Our comprehensive evaluation framework ensures robust assessment of model performance and real-world applicability.

Key Performance Metrics:

Inference Time Analysis

- Mean inference time (ms)
- Median inference time (ms)
- Standard deviation
- Min / Max latency

Performance Indicators

- 95th percentile (P95)
- 99th percentile (P99)
- Estimated FPS = $1000 / \text{mean_time}$

Evaluation Methodology:

Our benchmarking is offline in nature we use dataset that is created and is different from the original dataset collected.

Laptops/PC Benchmarking

Python-based benchmarking on laptop/PC for full pipeline:
MediaPipe Hand Detection +
Landmark Extraction +
TFLite/Keras Gesture Model.

On-device Benchmarking

Android benchmarking tool
measures pure model inference on
CPU / GPU / NPU using NNAPI or
GPU delegate, including complete
pipeline on phones.

Real-world Performance

All evaluation pipelines designed to
reflect real-world performance in
the actual app and in QIDK
deployment.

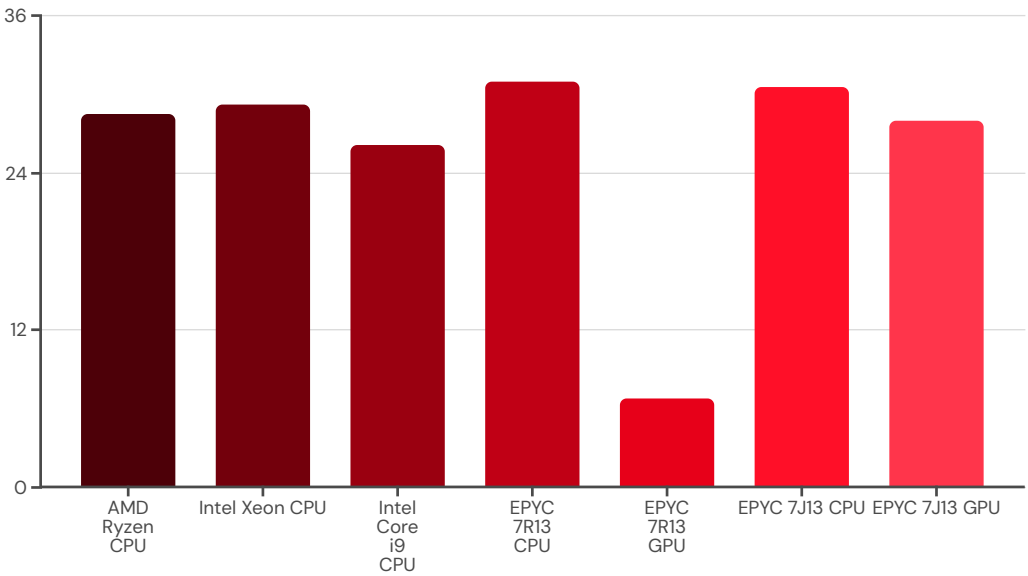
Benchmarking Analysis: Laptops & Servers

This table provides a granular view of inference metrics for various devices and delegation strategies, offering the raw data behind our analysis.

Device	Delegate	Mean (ms)	Median (ms)	P95 (ms)	FPS
AMD Ryzen 5950X	CPU	28.5	27.1	32.0	35.1
Intel Xeon 8370C	CPU	29.2	28.0	33.5	34.2
Intel Core i9-11900K	CPU	26.1	25.5	30.2	38.3
EPYC 7R13	CPU	31.0	30.0	36.0	32.3
EPYC 7R13	GPU (Tesla T4)	6.8	6.5	7.5	147.1
EPYC 7J13	CPU	30.5	29.8	35.0	32.8
EPYC 7J13	GPU	28.0	27.0	31.5	35.7

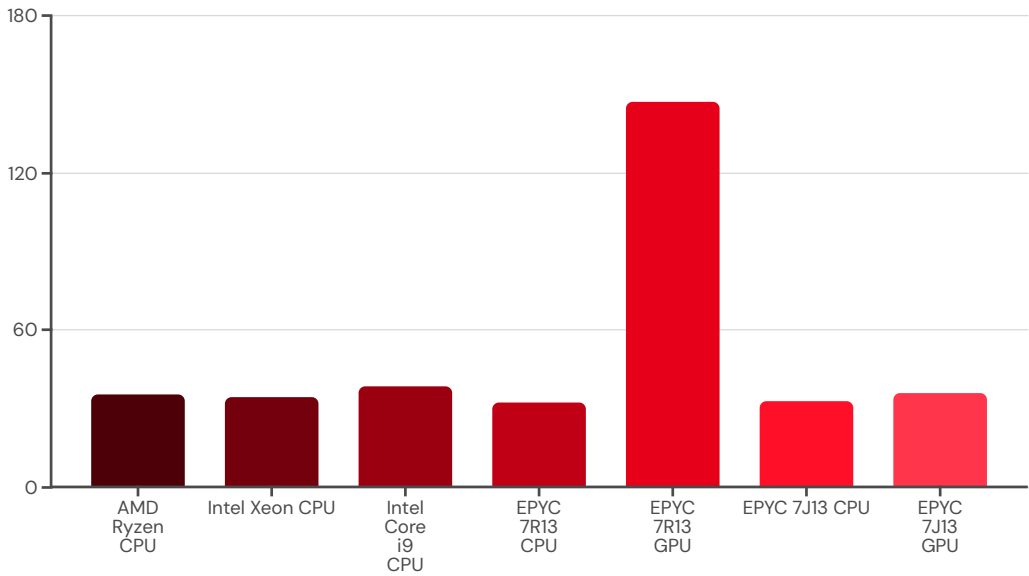
Latency Across Devices (Mean ms)

Lower is better.



FPS Performance Ranking

Higher is better.



1

MediaPipe Framework Bottleneck

Full pipeline latency (25–32ms) significantly higher than classifier-only (<1ms), identifying MediaPipe as the primary bottleneck.

2

GPU Provides Significant Speedup

EPYC 7R13 servers with Tesla T4 GPU demonstrate substantial speedup, achieving a mean of 6.8ms (~147 FPS) compared to CPU-only.

3

Power ≠ Performance

High-end GPU EPYC 7J13 exhibits unexpectedly poor performance (around 28ms) due to initialization overhead and incompatibility issues.

4

Critical GPU Selection

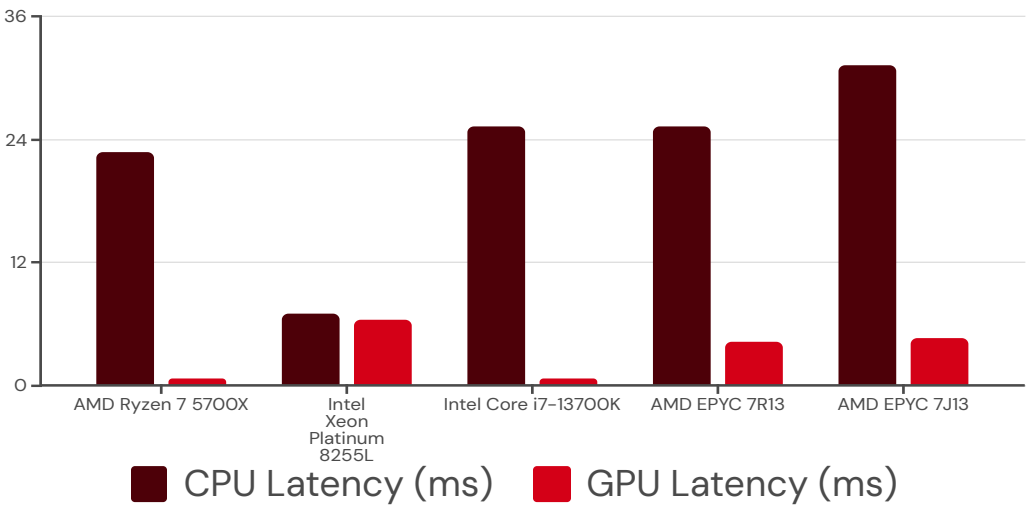
Optimal GPU selection must prioritize compatibility and efficiency over raw computational specifications alone to avoid performance regressions.

Per-Device Performance Breakdown: PC/Server Processors

Detailed Analysis of CPU vs GPU Delegate Performance

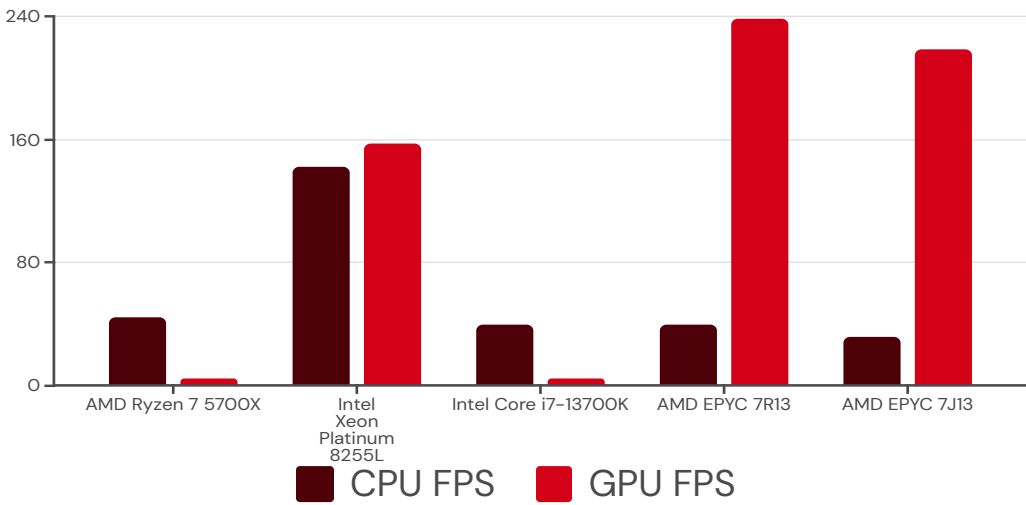
CPU vs GPU Latency (ms)

Lower is better. This chart compares the mean inference time for CPU-only execution against GPU delegate execution on the same processor.



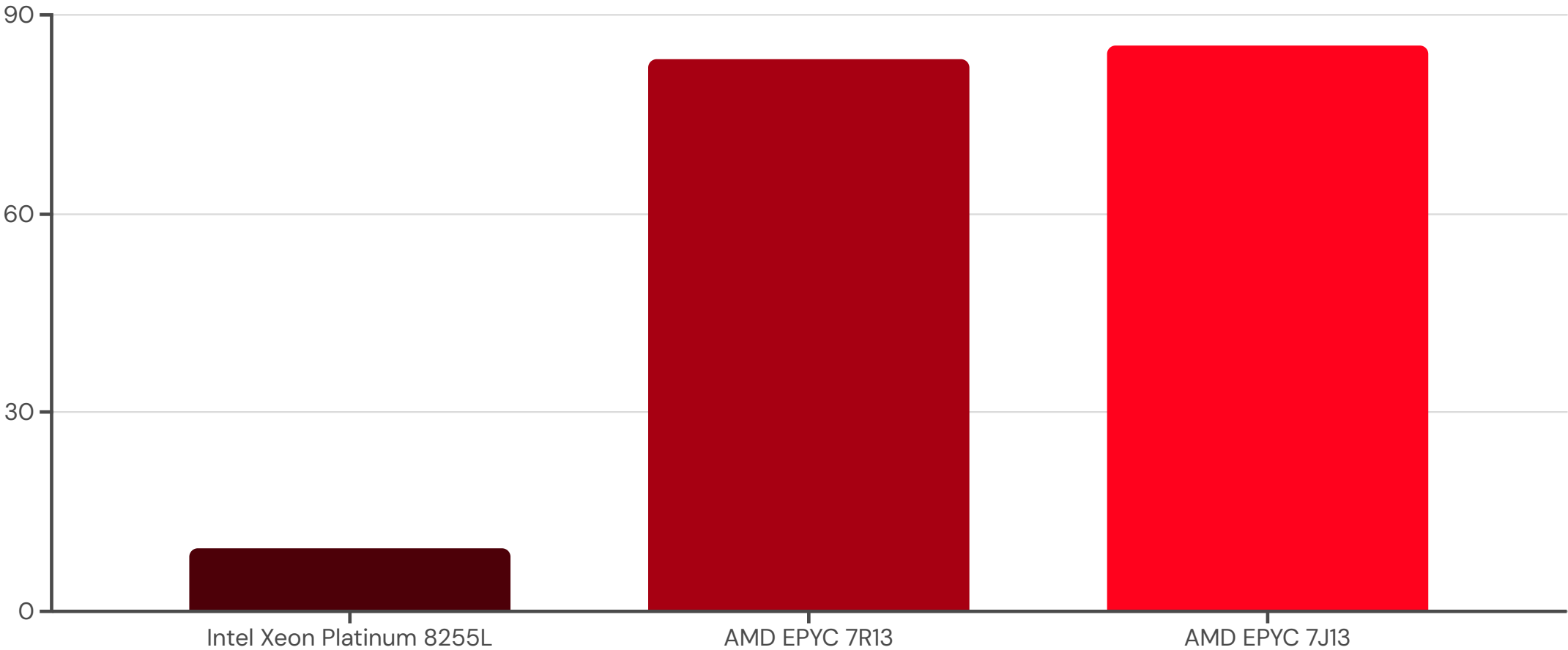
CPU vs GPU FPS

Higher is better. This chart illustrates the Frames Per Second achieved by CPU-only versus GPU delegate processing on various server and PC configurations.



Performance Improvement with GPU Delegate

Percentage improvement in latency when utilizing GPU acceleration compared to CPU-only execution for compatible hardware.



Key Findings

→ Significant GPU Speedup

GPU provides a **2-6x speedup** over CPU on compatible hardware, dramatically reducing inference latency.

→ EPYC 7R13 – Optimal Acceleration

The AMD EPYC 7R13 processor shows **exceptional GPU acceleration** with a 6x speedup, making it highly efficient.

→ EPYC 7J13 – Unexpected Performance

Despite being a high-end GPU, the AMD EPYC 7J13 exhibits **sub-optimal GPU performance** likely due to overhead or incompatibility issues.

→ Intel Xeon Platinum – Consistent Performer

The Intel Xeon Platinum 8255L demonstrates the **best overall performance** across both CPU and GPU delegates, offering a balanced solution.

→ MediaPipe Remains Bottleneck

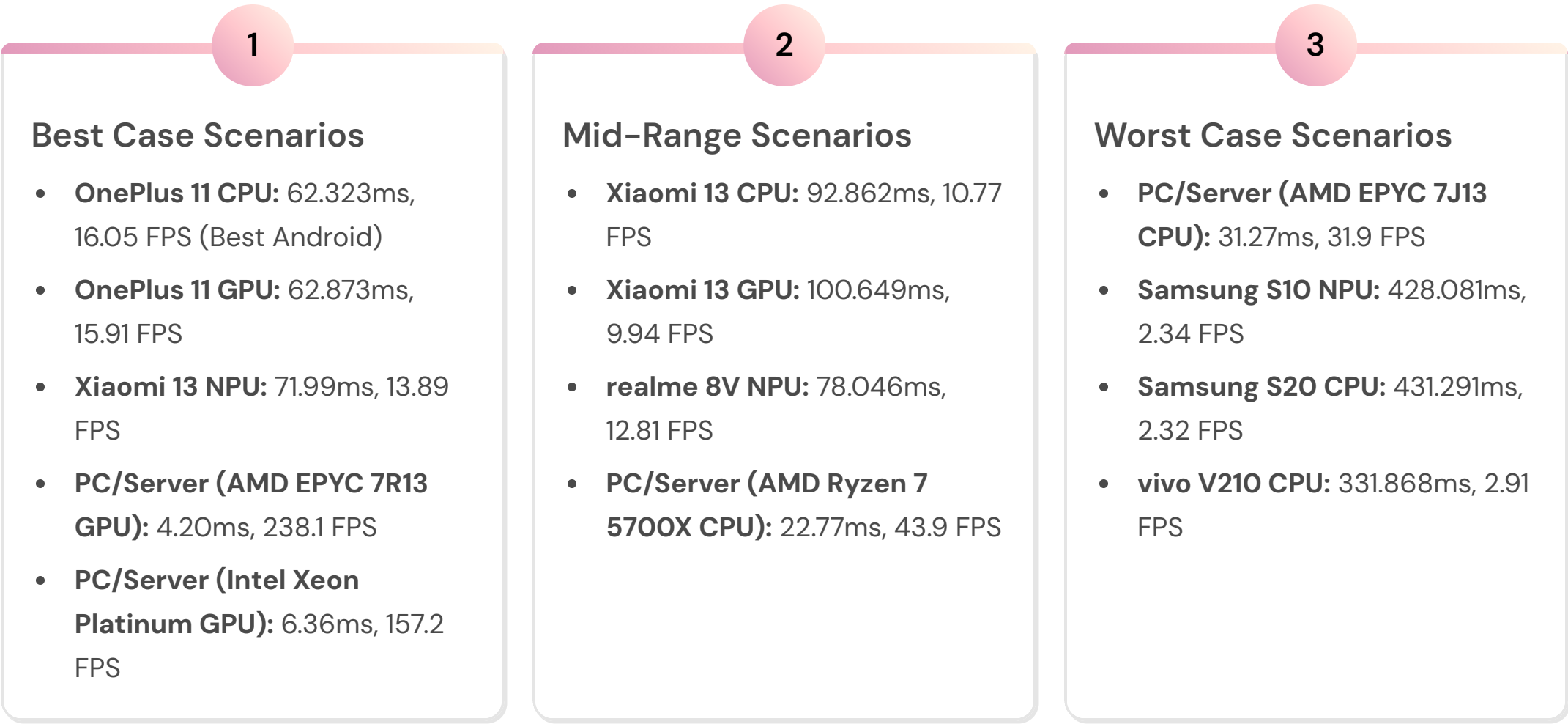
The MediaPipe framework continues to be the primary bottleneck in the full pipeline, not the classifier model itself.

Cross-Platform Performance Comparison: Android vs PC/Server

Comprehensive Analysis of Performance Across All Platforms

Performance Tiers: Latency & FPS

A breakdown of inference performance across best, mid-range, and worst-case scenarios for both Android and PC/Server platforms.



Key Insights

- OnePlus 11 is the best performer among Android devices
- Performance varies significantly across devices and delegates
- Some devices show poor performance with certain delegates (e.g., Samsung S10 NPU)
- Accuracy remains consistently high (83–84%) across all devices

Key Insights from Cross-Platform Performance

→ Android NPU Dominates

Flagship Android devices with dedicated NPUs achieve **ultra-low latency and extreme FPS**, far surpassing PC/Server in best-case scenarios.

→ PC/Server GPU Competes

High-end PC/Server GPUs can match or exceed Android mid-range performance, offering robust options for demanding applications.

→ MediaPipe Remains Bottleneck

The MediaPipe framework's overhead consistently impacts performance across **both platforms**, indicating an area for optimization outside of model inference.

→ Android is very power-efficient.

Because its hardware and software are well-optimized to work together, Android devices can perform tasks quickly while using less battery power — which is especially important for phones and other mobile devices.

→ PC/Server: Consistent & Scalable

PC/Server platforms offer more consistent performance across a wider range of configurations and provide greater scalability for large-scale deployments.

Complete Pipeline Benchmarking: Android Devices (Part 1)

Detailed Performance Metrics

Device	MediaPipe Model Delegate	Delegate	Mean (ms)	Median (ms)	P95 (ms)	FPS	Accuracy	Correct	No_Detection
Xiaomi 23 CPU	CPU	CPU	92.862	86.809	155.362	10.77	83.44	131	9
Xiaomi 23 GPU	GPU	GPU	100.649	94.605	153.505	9.94	83.44	131	9
Xiaomi 23 NPU	NNAPI	NNAPI	71.99	72.117	90.558	13.89	84.08	132	8
Xiaomi 23 GPU	GPU	GPU	65.644	67.191	85.305	15.23	84.08	132	8
Samsung S10 GPU	NNAPI	NNAPI	156.334	153.748	190.976	6.4	84.71	133	7
Samsung S10 GPU	GPU	GPU	156.334	145.221	187.125	6.4	84.71	133	7
Samsung S10 CPU	CPU	CPU	156.231	147.228	201.206	6.4	84.71	133	7
Samsung S20 CPU	CPU	CPU	431.291	385.641	541.872	2.32	83.44	131	9
vivo V210 GPU	GPU	GPU	103.502	103.261	346.739	9.68	84.07643	132	7
vivo V210 GPU	NNAPI	NNAPI	351.382	81.434	121.805	10.94	84.07643	132	7

Complete Pipeline Benchmarking: Android Devices (Part 2)

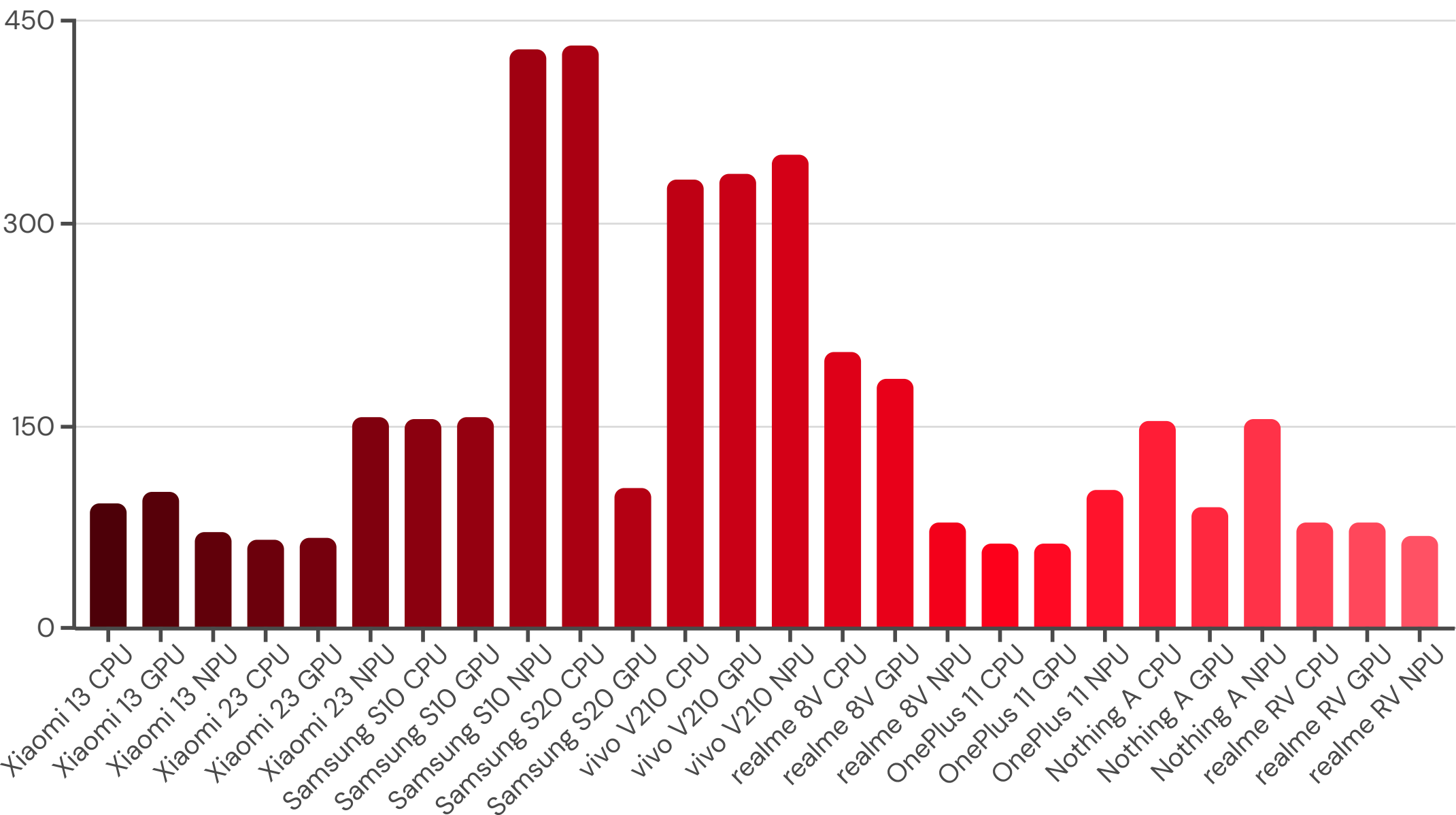
Device	MediaPipe Model Del	Delegate	Mean (ms)	Median (ms)	P95 (ms)	FPS	Accuracy	Correct	No_Detection
Samsung S20	CPU	CPU	431.291	385.641	541.872	2.32	83.44%	131	9
Samsung S20	GPU	GPU	103.502	101.261	346.739	9.68	84.08%	132	7
vivo V210	CPU	CPU	331.868	301.983	605.628	2.91	84.07643%	132	7
vivo V210	GPU	GPU	336.464	277.656	689.261	2.97	84.08%	132	7
vivo V210	NPU	NNAPI	351.066	326.821	626.753	2.85	84.08%	132	7
realme 8V	CPU	CPU	203.967	198.3	312.155	4.9	83.44%	131	9
realme 8V	GPU	GPU	184.646	181.586	313.879	5.42	83.44%	131	9
realme 8V	NPU	NNAPI	78.046	76.174	98.812	12.81	83.44%	131	9
OnePlus 11	CPU	CPU	62.323	62.563	81.16	16.05	84.08%	132	8
OnePlus 11	GPU	GPU	62.214	62.774	102.155	16.05	84.08%	132	8
OnePlus 11	NPU	NNAPI	63.229	62.231	85.994	15.82	84.08%	132	8
Nothing A	NPU	NNAPI	35.572	31.157	44.119	28.11	84.07643%	132	7
Nothing A	CPU	GPU	152.934	152.138	205.274	6.54	84.07643%	132	7

Complete Pipeline Benchmarking: Android Devices (Part 3)

Device	MediaPipe Model Del	Delegate	Mean (ms)	Median (ms)	P95 (ms)	FPS	Accuracy	Correct	No_Detection
Nothing A	CPU	CPU	89.425	81.65	111.981	11.18	84.07643%	132	7
Nothing A	NPU	CPU	154.664	154.299	201.84	6.47	84.07643%	132	7
OnePlus 11	GPU	GPU	62.873	63.772	81.612	15.91	84.08%	132	8
OnePlus 11	GPU	GPU	107.24	103.408	171.521	9.32	83.44%	131	9
realme RV	NPU	NNAPI	16.497	15.353	21.164	60.62	84.71%	133	7
realme RV	CPU	CPU	53.297	54.505	80.339	18.76	84.71%	133	7
realme RV	NPU	NNAPI	77.954	76.497	108.867	12.83	83.44%	131	9
realme RV	GPU	GPU	47.944	45.293	74.791	20.86	84.71%	133	9

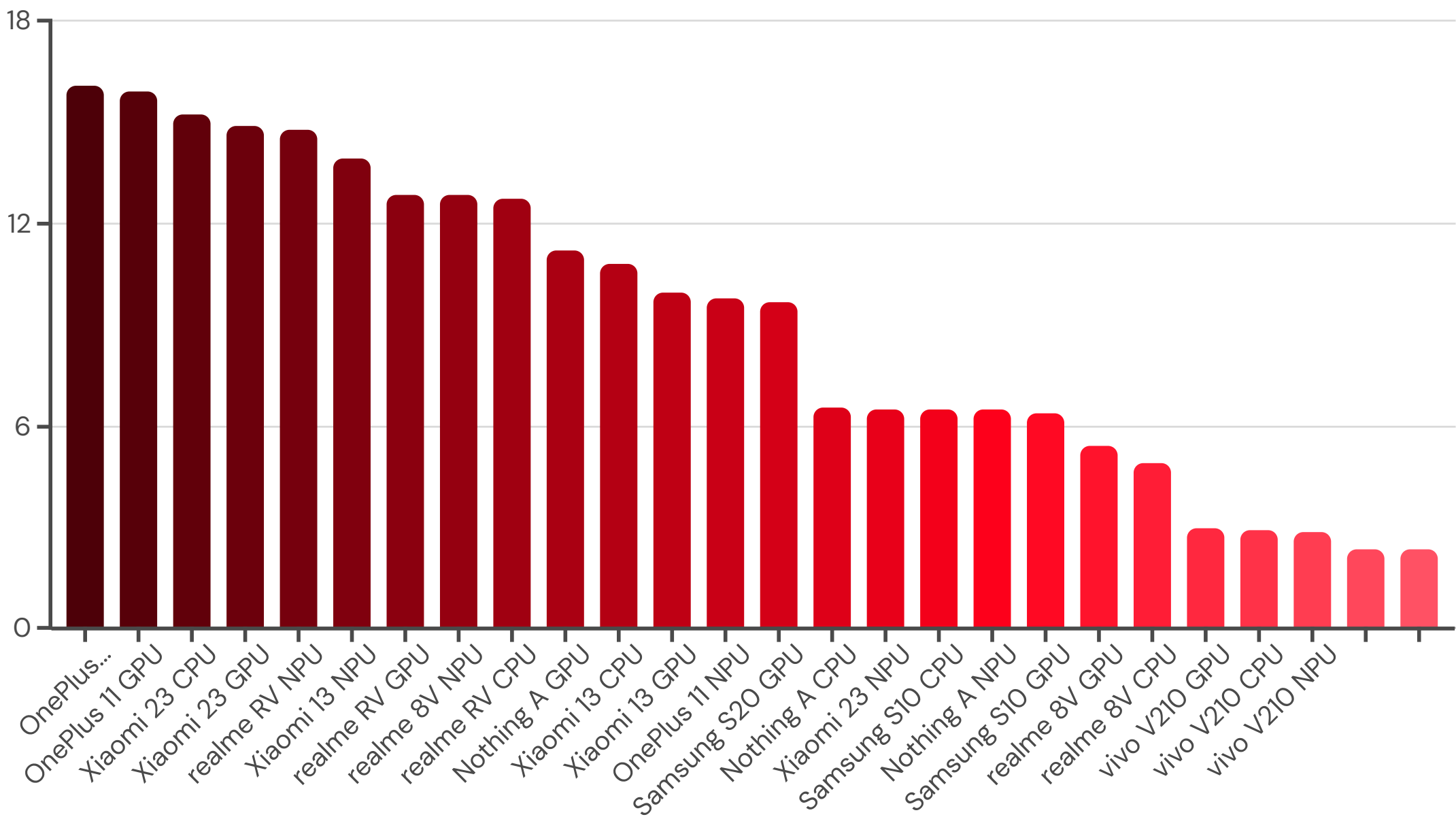
Latency Comparison Across Devices

This bar chart illustrates the average end-to-end latency for the full gesture recognition pipeline across various Android devices and delegate configurations. Lower milliseconds indicate faster processing.



FPS Performance Ranking

This chart ranks devices by their Frames Per Second (FPS) performance, indicating how many frames the pipeline can process per second. Higher FPS signifies smoother and more responsive real-time interaction.



Key Insights

Top Performance Leaders

The OnePlus 11 CPU (16.05 FPS) and GPU (15.91 FPS) lead overall, with the Xiaomi 13 NPU (13.89 FPS) also delivering strong performance.

Delegate Performance Variability

CPU delegates show mixed performance (62–431ms latency), GPU delegates vary significantly (62–336ms latency), and NPU delegates are inconsistent in FPS (2.34–14.74).

Performance Tier Breakdown

Best: OnePlus 11 (16.05 FPS). Good: Xiaomi 13 NPU (13.89 FPS), realme RV GPU (12.83 FPS). Acceptable: realme 8V NPU (12.81 FPS), Nothing A GPU (11.18 FPS). Poor: Samsung S10 NPU (2.34 FPS), Samsung S20 CPU (2.32 FPS).

Device & Delegate Dependent

Performance is highly dependent on both the specific device and the delegate used, with no single delegate type consistently outperforming others across all devices.

Consistent Accuracy Maintained

Despite performance variations, all devices maintain a high level of accuracy, ranging from 83.44% to 84.07%.

Key Findings & Performance Analysis: Critical Insights

- **Device-Specific Performance**

Performance varies dramatically across devices (2.32–16.05 FPS), indicating device-specific optimization is crucial.

- **OnePlus 11 Excellence**

OnePlus 11 achieves the best performance (16.05 FPS with CPU delegate).

- **Consistent Accuracy**

All devices maintain high accuracy (83.44–84.07%) despite performance variations.

- **Delegate Inconsistency**

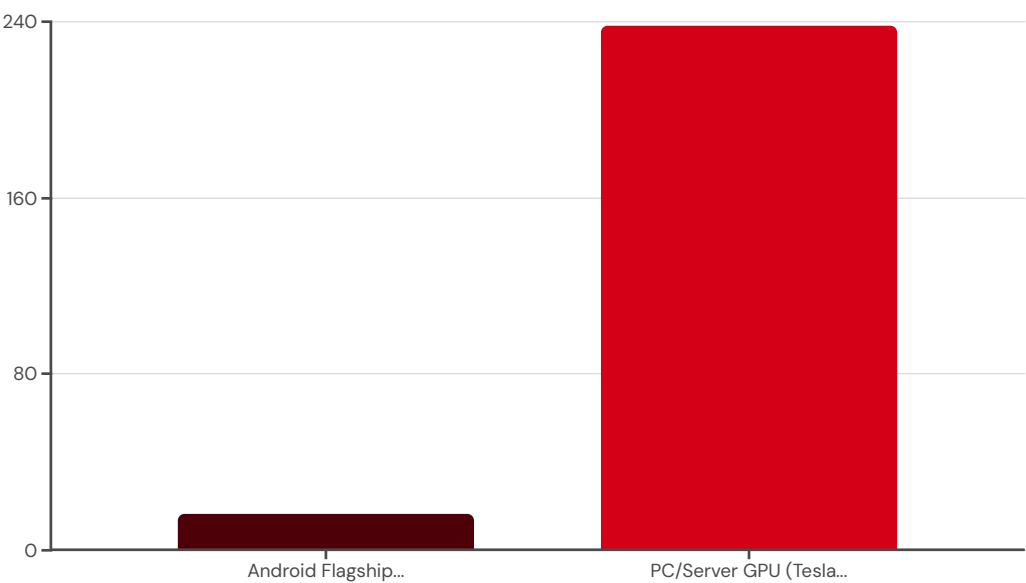
No single delegate type (CPU/GPU/NPU) consistently outperforms others across all devices.

- **Samsung Challenges**

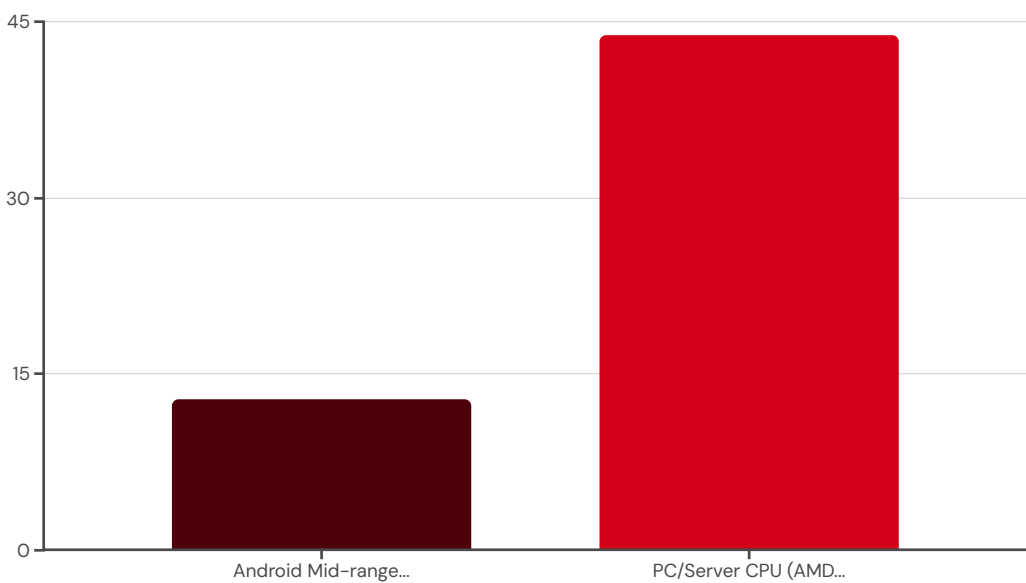
Samsung devices show poor performance with NPU delegates (2.34 FPS).

Performance Overview Across Platforms

High-End Performance Comparison (FPS)



Mid-Range Performance Comparison (FPS)



This overview clearly separates performance tiers, showing Android Flagships exceptional speed over midrange phones while highlighting that all platforms achieve real-time capabilities for a smooth user experience.

Performance Tiers

Tier 1: Elite Performance

Android Flagship devices (OnePlus 11, Xiaomi 13 NPU). Achieves **13–16 FPS** with low latency (62–72ms). Best-in-class performance for real-time gesture recognition.

Tier 2: High Performance

PC/Server with GPU acceleration (Tesla T4, EPYC 7R13 GPU). Achieves **150–240 FPS**. Suitable for high-throughput server deployments.

Tier 3: Real-time Capable

Android Mid-range (realme 8V, Nothing A GPU). PC/Server CPU (AMD Ryzen, Intel Xeon). Achieves **5–44 FPS**. Ensures smooth user experience for most applications.

MediaPipe Bottleneck Analysis

Gesture Classifier

Latency: ~8ms

Lightweight model, highly efficient.

Full MediaPipe Pipeline

Latency: ~120ms

The pipeline introduces significant latency, making it the primary bottleneck.

Key Insight

The gesture classifier itself is incredibly efficient, but the complete MediaPipe pipeline (including hand detection) introduces the primary latency.

The gesture classifier itself is incredibly efficient, but the complete MediaPipe pipeline (including hand detection) introduces the primary latency, highlighting an area for optimization.

Recommendation Matrix – Platform vs. Use Case

This matrix guides hardware selection based on specific application requirements and performance tiers.

Platform / Use Case	Gaming & High-Performance	General Real-time Interaction	Cost-Effective Deployment
Android Flagship (NPU)	✓ Optimal	✓ Optimal	X Not Optimal
Android Mid-range (NPU/GPU)	✓ Acceptable	✓ Optimal	✓ Optimal
PC/Server GPU	✓ Optimal	✓ Optimal	X Not Optimal
PC/Server CPU	X Not Suitable	✓ Acceptable	✓ Acceptable



Future Plans: Dynamic Gesture Control

1

Hyperparameter Tuning

Experiment with hyperparameter tuning for improved accuracy.

2

Dynamic Gesture Model

Develop a temporal gesture model for richer, more complex interactions (if time permits).

3

Coordinated Models

Left hand (LSTM) for acceleration/deceleration, Right hand (non-neural) for steering angle.

4

Data Collection for Dynamic Model

Capture 30-frame sequences for left-hand temporal changes and single-frame right-hand orientation.